

On the Use of Requirements Measures to Predict Software Project and Product Measures in the Context of Android Mobile Apps: a Preliminary Study

Rita Francese*, Carmine Gravino*, Michele Risi*, Giuseppe Scanniello[†], and Genoveffa Tortora*

*DISTRA (MIT), University of Salerno, Italy

Email: {francese, gravino, mrisi, tortora}@unisa.it

[†]DiMIE, University of Basilicata, Italy

Email: giuseppe.scanniello@unibas.it

Abstract—In this paper, we study the value of software project and product measures in the context of Android mobile apps. In particular, we focus on the effort to develop mobile apps and the number of graphical components in these apps. Estimation models are based on information from requirements specification documents (e.g., number of actors, number of use cases, and number of classes). We have used a dataset containing information on 23 Android apps and employed a stepwise linear regression to build estimation models. The predictions have been compared with those obtained considering models built on software measures (e.g., number of classes, number of files, and number of line of code). The results suggest that the measures from the artifacts produced in requirements engineering process are not worse predictors than those measures from source code. That is, requirements measures can effectively employed to estimate software project and product measures of a mobile app and estimations can be done early in the software development process.

Keywords—Empirical study; mobile app development; requirements measures; software development effort estimation.

I. INTRODUCTION

Software estimation consists in predicting the amount of effort (expressed in terms of person-hours or money) to develop or maintain software based on incomplete, uncertain, and noisy input. Estimation models can also concern different project measures. Therefore, estimates may be used into project plans, iteration plans, budgets, investment analyses, pricing processes, bidding rounds, and any project software artifacts [1].

In the literature, a number of software development effort estimation approaches have been proposed [1]–[3]. There are many proposals to classify these approaches (e.g., [2]). Independently from these classification proposals, effort estimation approaches fall into the following three top level categories: (i) expert estimation, an estimate is produced based on judgemental processes made by software experts; (ii) formal estimation model, the quantification step is based on mechanical processes such as a mathematical model derived from historical data; and (ii) combination-based estimation, it is based on a judgemental and mechanical combination built on different sources of information.

The state of the art evidences that there is not the *best approach* for software development effort estimation because of the huge differences in the estimation accuracy [3]. In addition, the relative accuracy of one approach or model in comparison to others strongly depends on the software project context [4]. The technology to be used in the development of a given software might also play an important role in estimation accuracy [5]. For example, it could be possible that a technique is accurate to estimate the effort to develop traditional desktop applications, while it does not properly work in the case of different kinds of applications such as web applications and mobile apps.

As for formal estimation models, the differences in the estimates may be also caused by the kind of software artifacts employed to build these models. Models built on software artifacts produced in the early phases of the development process could be more useful than those built on software artifacts produced later in the development process, but less accurate in prediction.

In this paper, we address the problem of comparing software project and product measures in the context of mobile app for Android devices. We built formal estimation models for the effort to develop mobile apps and the number of components the GUI (Graphical User Interface) of these apps contain. Estimation models are based on information from requirements specification documents. For example, we considered the numbers of actors, use cases, and classes of the conceptual model of a the app to perform estimations early in its development process. We have considered 23 Android mobile apps and employed a stepwise linear regression technique to build estimation models. We also verified if requirements measures are effective as number of classes, number of files, and number of line of code in the estimation of software project and product measures.

Paper Structure. In Section II, we introduce the basic concepts and definitions in the context of native app development for Android devices. The design of our empirical study is shown in Section III. We present and discuss the obtained results in Section IV. Final remarks of our research conclude this paper.

II. BACKGROUND

One of the mobile app challenges is to deal with multiple platforms during mobile development [6]. Developers can create mobile apps by using either native development tools for each of the major mobile platforms, such as iOS, Android, Microsoft Windows Mobile, Symbian, BlackBerry, or cross-platform environments, including PhoneGap and Titanium. At the present, developers separately create the app for each platform. Indeed, the features of a specific operating system may not be available in another. Alternatively, developers can develop a cross-platform app that runs on any environment, but has more limited functionalities. For example, to create an app that exploits in the better way the features of an Android device, developers have to master development skills related to the Android operating system and the associated development environment and resources. As for the available resources, smartphones are equipped by sensors, such as accelerometer, gyroscope, GPS, brightness and temperature, offer communication features, including phone calls, SMS, email, and camera functionalities.

The main Android components are: the activity and the service. An activity is an app component that provides a screen with which users can interact in order to do something, such as dial the phone, take a photo, send an email, or view a map. Each activity is given a window in which to draw its user interface. For simplicity reason, we will refer to an activity as a GUI. A service is a background component that performs either long-running operations or works for remote processes. A service does not provide a user interface. For example, a service might fetch data over the network without blocking the interaction the user has with an activity.

Developers can create the GUI of an Android activity directly in Java or by using an XML-based layout file. The latter approach has two main advantages. It allows to 1) separate logic from presentation; 2) to maintain different parallel layouts for difference screen sizes.

III. EMPIRICAL INVESTIGATION

In this section, we present the design underlying our empirical investigation.

A. Definition

Using the Goal Question Metrics (GQM) template [7], the goal of our study can be defined as follows:

- **Analyze** the use of measures obtained from the requirements and analysis document,
- **for the purpose of** building prediction models,
- **with respect to** the development effort and the number of graphical components of a GUI mobile app,
- **from the point of view** of the researcher and the professional developer,
- **in the context of** mobile apps developed by young developers grouped in small teams.

The GQM formalism allowed us to define important aspects before the planning and the execution of the study took place [8].

Accordingly to the goal of our study, we have formulated and investigated the following research questions:

- RQ1.** Do the measures obtained from the requirements and analysis document provide accurate predictions of effort of mobile applications?
- RQ2.** Do the measures obtained from the requirements and analysis document provide accurate predictions of the number of graphical components of mobile applications?

We have selected a baseline approach to evaluate the accuracy of our estimations. The most natural baseline is represented by the predictions achieved by using software measures from source code. Software metrics have been gathered by Understand.¹ The rationale behind the choice of this baseline is to verify that the measures obtained early in the development process do not perform significantly worse than measures obtained later in the development process in the context of our predictions for mobile apps.

B. Context

The empirical investigation has been conducted on 46 Bachelor students in Computer Science at the University of Salerno. The students were enrolled to a Mobile Application Development course. This course was focussed on the design and development of mobile apps for the Android operating system. The course duration was of 12 weeks (48 hours).

Within the Mobile Application Development course the students had to develop mobile apps as a mandatory laboratory activity. In particular, the students proposed, designed, and developed a native mobile app for Android. Each proposal was accurately motivated by proponents. To this end, a market analysis was also conducted and the results were presented to the lecturer. The apps had to respect the following nonfunctional requirements: the app had to interact with a remote server, through JSON. The lecturer imposed the nonfunctional requirements. Each mobile app had to exploit native device functionalities, including maps, GPS, sensors, phone call and SMS. It had to manage the device rotation, and use SQLite. Games were admitted if they exposed back-end functionalities, such as account management, multiuser support, bonus management, and the upgrade of the app. The development started after the approval of the lecturer. The approved apps fell in different application domains: game, social networking, geolocalized services, sports, entertainment, travel, restaurant, transport, public administration, and people management.

Participants were grouped in 23 teams. In particular, 2 teams were composed of 1 student, 19 teams were composed of 2 students, and 2 teams were composed of 3 students. The

¹scitools.com

team composition was freely chosen by the students. We did not decide to assign randomly team members because the students had previous experience of project work in several courses and, at the last term, they know which are the classmates more appropriate for them. The students were asked to use GitHub [9] for the management activities. The lecturer creates a GitHub account for each group. The templates to document design and development activities were made available in the GitHub repository of each group of students.

We established a schedule for each team of students. They were informed about deadlines and deliverables. The first deliverable was the project proposal, the second the Requirements Analysis Document (RAD). Successively, the participants had to deliver the different releases of their mobile app and the final version of this app. As for the requirements specification documents, the participants were asked to use the template by Bruegge and Dutoit [10].

We asked the students to follow an incremental prototyping development approach. The students have to show three app prototypes to the lecturer before the conclusion of project. We also promoted the adoption of pair programming, an agile software development technique in which two programmers work as a pair together on one workstation. These choices were taken to simulate an actual software development project.

The participants in the study were asked to fill in a pre-questionnaire to collect some demographic information and development experiences. In particular, we asked questions about: the use of Android device (as smart-users), the development of Java, Web, and Android apps, the knowledge of software engineering and database technologies, and the experience in network programming. The answers to the questions allowed us to observe that the students were generally smart-users for mobile apps, they did not know Android development and that most of them stated to be good Java developers and to have database development experience. 30 and 33 participants affirmed to have experience in applying software engineering methods and approaches to design and develop desktop and web applications, respectively. On the other hand, 14 participants declared to have good network programming experiences.

C. Planning

Our investigation used information from RADs for predicting the effort for developing mobile apps and their graphical components. In particular, size measures from these documents (e.g., use case diagrams and class diagrams) were correlated with the effort. We postulated that the effort is proportional, on a first approximation, to the number of actors, functional requirements, use cases, and classes specified in the RAD. It is worth remarking that a RAD is produced in the early phase of a development process. This is important because predictions should be made as soon

Table I
DEPENDENT VARIABLES

Measure	Description
Effort	The total effort to develop a mobile app expressed in terms of person/hour
XMI_UI	Number of XMI files about graphical elements (e.g., text box and so on) of a mobile app

Table II
VARIABLES DENOTING INFORMATION FROM REQUIREMENTS AND ANALYSIS DOCUMENTS

Measure	Description
FR	Number of functional requirements
Act	Number of use case actors
UC	Number of use cases
Cla	Number of classes
SD	Number of sequence diagrams

as possible in the development process. As for the number of graphical components of a mobile app, they could be related to information included in the RAD. We used size measures from this kind of documents to build prediction models to estimate the number of graphical elements of the GUI of mobile apps. This choice would make little sense in the context of desktop applications, but it could be not true in the context of mobile apps. In fact, mobile apps are mostly used to provide a smart back-end to any kind of users: experienced or not.

D. Selected Variables

The considered dependent variables are: Effort² and XMI_UI. We summarized the meaning of these variables in Table I. Effort is used for studying the first research question, while XMI_UI is employed for the second. We considered two sets of independent variables. They are reported and described in Table II and Table III, respectively. The choice of using these two set of variables was needed because we had to compare prediction accuracy of software measures obtained from RADs (RAD measures, from here on) against the accuracy of the predictions obtained with source code measures (SC measures, from here on).

E. Data Analysis

1) *Estimation Techniques*: In our investigation, we employed the StepWise Linear Regression (SWLR) technique. SWLR allows computing linear regression in stages. We opted for linear regression because it has been widely used in the context of software prediction with appreciable results [3], [12]–[14].

SWLR explores the relationship between a dependent variable and one or more independent variables, providing

²The time represents an approximation for effort. This is almost customary in literature and it is compliant with the ISO/IEC 9126 standard [11], where effort is the productive time associated with a specific project task.

Table III
VARIABLES DENOTING INFORMATION FROM SOURCE CODE OBTAINED
BY THE UNDERSTAND TOOL

Measure	Description
McB	McCabe Cyclomatic complexity
Classes	Number of classes
Files	Number of files
Methods	Number of methods, including inherited ones
NL	Number of all lines
LOC	Number of lines containing source code
CLOC	Number of lines containing comment
STM	Number of statements

a model described by a linear equation:

$$y = b_1x_1 + b_2x_2 + \dots + b_nx_n + c$$

where y is the dependent variable, $x_1, x_2, \dots, x_n$ are the independent variables, b_i is the coefficient that represents the amount variable y changes when variables x_i changes 1 unit, and c is the intercept.

SWLR allows computing an equation in stages in which the choice of the independent variables is carried out by an automatic procedure. These variables can be chosen applying three approaches: forward, backward, or a combination of both [15]. The forward approach starts with no variables in the model. It tries out the variables one by one and includes them in the model if they are statistically significantly correlated with the dependent variable. The backward approach starts with all the variables and test them one by one. We remove the variables that are not statistically significant correlated with the dependent variable. We used here a combination of forward and backward approaches. At each step, this combined approach includes or removes variables one by one if they are or not statistically significant correlated with the dependent variable.

To evaluate the goodness of fit of a model, several indicators have been proposed. Among them, we exploited the square of the linear correlation coefficient (i.e., R^2), that shows the amount of variance of the dependent variable explained by the model related to an independent variable. A good model should be characterized by a high R^2 value. We also considered the F value indicators and the corresponding p-value (denoted by Sign. F), whose high and low values, respectively, denote a high degree of confidence for the prediction.

2) *Prediction Validation*: To assess the predictions of the models, we performed a k-fold cross validation. This kind of validation is widely used to assess how the results of a statistical analysis can be generalized to an independent dataset [16]. In particular, when the goal is the prediction, the k-fold cross validation is used to estimate how accurately a predictive model will perform in practice. The validation process performs k rounds. Each round involves the splitting of the original dataset into training and test sets. The

training set is used to build an estimation model, while the test set to validate that model. The results are averaged over the k rounds. In particular, we exploited a leave-one-out cross validation, where $k = n$ and n is the size of the dataset. Thus, the dataset is divided into n different subsets of training and test sets. Each test set contains only one observation. This approach is almost common in the estimation field (e.g., [17]).

To evaluate the accuracy of the obtained estimations, we computed: median (Md) and mean (M) of Absolute Residuals (AR). Given the predicted value pre and the actual value act , the absolute residual is equal to $|act - pre|$. This is a widely used performance measure [18]. The smaller the value, the better the prediction is. That is, the actual and predicted values are very close one another.

To answer our research questions, we compared the MAR and MdAR values obtained by using the prediction model built exploiting RAD measures with those achieved by applying the prediction model built exploiting SC measures. We also exploited boxplots to graphically summarize the distributions of absolute residuals.

Finally, we applied the non parametric Wilcoxon statistical test [19] to verify whether there was statistically significant difference between the estimations achieved with the models based on RAD and SC measures. This test allows us to verify whether the absolute residuals obtained with the SWLR model employing RAD measures are not significantly different from those achieved by employing SC measures. We opted for this test because it is very robust and because it has been widely applied in statistical analyses similar to that we performed in this research work. In addition, we expected that data were not normally distributed. This assumption was properly verified by means of the Shapiro test [20].

For all the statistical tests performed, we decided to accept a probability of 5% of committing a Type-I-Error [8].

F. Threats to Validity

To comprehend strengths and limitations of our study, threats that could affect results and their generalization are presented and discussed. Despite our effort in mitigating as many threats as possible, some of them are unavoidable.

The *external validity* threats are always present when exploiting data from a specific context. We mitigated this threat considering mobile apps from different application domains. However, replications with other systems belonging to different domains are needed. Threats to external validity are also related to the complexity and the size of the apps in our empirical study. These apps are not far from those that users can download from a market place (e.g., Google Play). Finally, the use of students may also affect this kind of validity [21]. However, mobile apps are very often developed by people with a low programming

experience. In addition, it is very difficult to find also professional programmers with a high programming experience on mobile development technologies. This is due to the fact that these kinds of technologies are not adequately mature and therefore it is difficult to find people skilled on them also in the software industry. Therefore, we are happy to believe that the use of students is not a major issue here given also that a few of them had a small experience as professional programmers. Also, the constraints imposed to the students to develop mobile apps represent another threat to the validity of the results. For example, we imposed to define a RAD and use it as the basis for the development of mobile apps. This practice could be not adopted in industry, so raising some concerns on the representativeness of our study. Since there are not well established practices in the design and development of mobile apps, our design choice is not a major issue for external validity. Another threat to external validity is the team composition. In industry, team composition is not based on professional preferences rather than on project needs.

The *conclusion validity* threats concern issues that affect the ability of drawing correct conclusions. In our context, this kind of validity refers to a statistical inference from a sample to a study population [22]. The used evaluation criteria allowed assessing in an objective way the effectiveness of the predictions. Proper non parametric statistical tests (i.e., the Wilcoxon test) were also used. The number of observations could affect the validity of the conclusions. Thus, replications with a larger dataset are needed.

Internal validity regards the used experimental procedure. The treatments or the experiences of the participants might also affect this kind of validity. In this study, we did of our best to control all the possible “extraneous” factors. For example, we used guidelines to conduct the experiment and to analyze the data [8].

A threat for the *construct validity* concerns the ability of establishing a correct operational measure for the concepts considered in the empirical analysis [23]. In our case, other independent variables could exist and they could be considered in our prediction models. This point is the subject of future work and our study will pose the basis in such a possible direction.

IV. RESULTS AND DISCUSSION

Some descriptive statistics (i.e., minimum and maximum values, mean, median, and standard deviation) of the independent variables are shown in Table IV. For the dependent variables, descriptive statistics are also reported.

Before applying SWLR, we verified the following assumptions: (i) the existence of a linear relationship between the independent and the dependent variables (i.e., linearity), (ii) the constant variance of the error terms for all the values of the independent variable (i.e., homoscedasticity), and (iii) the normal distribution of the error terms (i.e.,

Table IV
DESCRIPTIVE STATISTICS OF THE VARIABLES

Variable	Min	Max	Mean	Med	St.Dev.
FR	4	23	8.481	8	4.291
Act	1	4	1.593	1	0.797
UC	4	26	10.778	8	5.8
Cla	10	57	21.7778	19	12.1
SD	3	16	7.074	6	3.025
McB	48	4030	517.519	282	747.912
Classes	12	967	89.222	54	178.623
Files	5	273	34.185	23	50.157
Methods	192	15222	1510.074	943	2795.714
NL	534	42287	5134.556	2740	7854.716
LOC	258	29456	3599.926	2037	5455.17
CLOC	12	3108	393.556	258	591.7
STM	163	21369	2714.444	1464	3969.109
DIT	2	4	2.444	2	0.577
Effort	30	113	58.815	55	21.042
XMI_UI	8	105	33.407	30	24.262

Table V
RESULTS OF SWLR FOR EACH DEPENDENT VARIABLE USING THE RAD MEASURES

Dependent variable	Independent variables	R ²	F	Sign. F (p-value)
Effort	Act	0.233	3.65	0.041
	Cla			
	Intercept			
XMI_UI	UC	0.424	8.85	0.001
	Cla			
	Intercept			

normality). We performed a log transformation of the input variables because the RAD measures were not normally distributed according to the results of the executed Shapiro test. Furthermore, we performed the analysis of outliers, exploiting the Cook’s distance and performed a stability analysis as suggested by Mendes and Kitchenham [24] to eliminate influential observations.

As for the two sets of independent variables employed in our analysis, the results of the performed SWLR are summarized in Table V and Table VI, respectively. We can observe that the models built using RAD measures are characterized by a Sig. F value less than 0.05, thus the resulted model is significant. However, the obtained R² and F values are not so high. As for the models based on the use of SC measures, we note that the model considering Effort as dependent variable is not characterized by a Sign. F value less than 0.05. Again, the obtained R² and F values are not so high for both the models.

For both the models built using RAD measures, the variables selected as best effort predictors include Cla. A plausible justification for this outcome is that the number of classes in a requirements specification document represents the basis for the next phases of the development process.

Table VI
RESULTS OF SWLR FOR EACH DEPENDENT VARIABLE USING THE SC MEASURES

Dependent variable	Independent variables	R ²	F	Sign. F (p-value)
Effort	Classes	0.202	3.03	0.067
	NL			
	Intercept			
XMI_UI	McB	0.706	13.2	<0.001
	LOC			
	STM			
	DIT			
	Intercept			

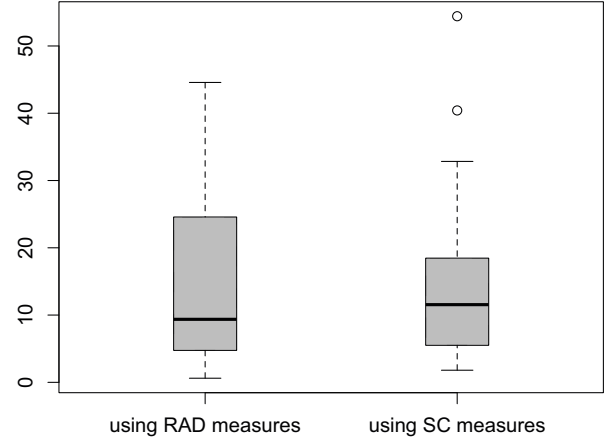
Table VII
RESULTS IN TERMS OF MAR AND MdAR OBTAINED BY THE BUILT SWLR MODELS

Dependent variables	Independent variables	MAR	MdAR
Effort	Act, Cla	15.63	9.37
	Classes, NL	14.34	11.55
XMI_UI	UC, Cla	12.15	8.41
	McB, LOC, STM, DIT	9.81	8.76

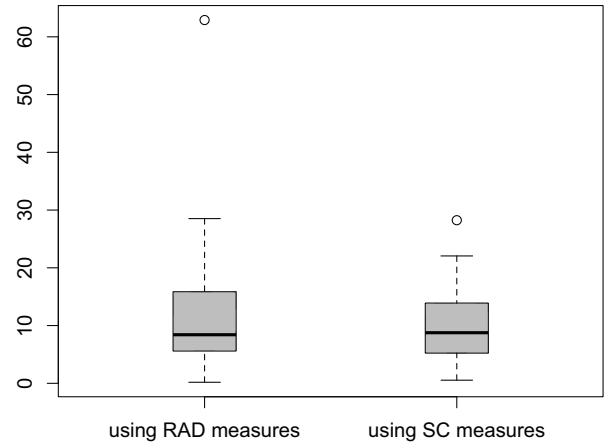
That is, a developer uses these classes as the starting point for development. Therefore, it seems reasonable that Cla provides useful information for an accurate prediction of both the size of app GUI and the effort to develop these apps. As for the XMI_UI predictors, we can justify the selection of UC because in mobile app development there is a relationship between use cases and GUIs. In the apps of our dataset, we observed that the number of use cases that do not have any associated GUI is limited (i.e., the services). In other words, mobile apps are devised to present data rather than to perform their manipulation. Mobile apps provide a back-end to access information, while computation is performed elsewhere and invoked as a service.

On the other hand, the model built on SC measures to predict Effort includes as predictors the variables Classes and NL, while the model built to predict XMI_UI includes the variables McB, LOC, STM, and DIT. As for Effort, it could make sense because the effort to develop any applications (both traditional and mobile) should be proportional to the number of classes and total number of statements and comments. It is worth noting that the model for XMI_UI included variables that seem to be not related with SC measures. This could be related to the fact that GUIs in mobile apps are not defined in source code, but in XML files. This point deserves further investigations and our study poses the basis in this direction.

Table VII shows the results in terms of MAR and MdAR achieved employing SWLR models built using both RAD measures and SC measures. When predicting Effort, we can observe that the value of MAR obtained with the model based on Act and Cla variables is close to the MAR values



(a) Predicting Effort



(b) Predicting XMI_UI

Figure 1. Absolute residuals obtained by the built SWLR models

obtained by using Classes and NL. Furthermore, the value of MdAR achieved by the model based on RAD measures is even better than the value of MdAR achieved by the model based on SC measures.

As for the dependent variable XMI_UI, the value of MAR obtained by the model based on UC and Cla variables is slightly worse than the MAR value obtained by using McB, LOC, STM, DIT. Differently, the value of MdAR achieved by the model based on RAD measures is even better than the MdAR value achieved by the model based on SC measures.

The results before are further confirmed by the boxplots of absolute residuals reported in Figure 1. Indeed, we can observe that when predicting Effort, even if the box length and tails of the boxplot for RAD measures are more skewed than the ones of the boxplot for SC measures, it has no outliers. Furthermore, the median is closer to zero. In case of XMI_UI, the box length and tails of the two boxplots are very close. The boxplot for RAD measures has one outlier more far than the one of the boxplots for SC measures.

Table VIII
RESULTS OF MANN-WHITNEY TEST TO VERIFY STATISTICALLY
SIGNIFICANT DIFFERENCES AMONG ACHIEVED ABSOLUTE RESIDUALS
OBTAINED BY SWLR_{RAD} AND SWLR_{SC}

Dependent variable	p-value	Significant difference?
Effort	0.985	No
XMI_UI	0.773	No

For each dependent variable, Table VIII shows the results of the Wilcoxon test. The obtained results suggest that there was not a statistically significant difference among the absolute residuals obtained by the RAD measure based model (named SWLR_{RAD} in table) and the absolute residuals achieved by the SC measure based model (named SWLR_{SC} in table).

On the basis of the results presented and discussed before, we summarize our outcomes with respect to the defined research questions:

- RQ1.** The measures obtained from the requirements and analysis document provide accurate predictions of the effort needed for mobile applications developed in small teams (from 1 to 3 members), comparable with those achieved using measures obtained from source code. This implies that this research question can be positively answered.
- RQ2.** The measures obtained from the requirements and analysis document provide accurate predictions of the number of graphical components of mobile applications, comparable with those achieved using measures obtained from source code. Also, this research question can be positively answered.

V. CONCLUSION

Estimation in the context of mobile development has been marginally investigated [25]–[27]. For example, Souza and Aquino discussed the applicability of traditional estimation models for the purpose of developing systems in the context of mobile computing. The used empirical strategy is the systematic literature review. On the other hand, Nitze *et al.* [26] proposed an analogy-based effort estimation approach for mobile app development. In particular, they proposed different techniques to estimate size, effort, and cost for developing mobile apps. Similarly, Heeringen and Van Gorp [27] proposed an approximation of the COSMIC method [28] to estimate the functional size of mobile apps. More recently, D’Avanzo *et al.* [29] proposed guidelines to be employed together with the standard COSMIC method to measure functional sizes of mobile apps. These sizes have been employed to estimate mobile applications code sizes.

Unlike the papers discussed before, we have presented the results of a preliminary study conducted to study the value of software project and product measures to estimate the effort needed to develop mobile apps as well as the number

of graphical components in the GUIs in the context of small/medium mobile apps. The estimation models are built on information gathered in the requirements specification documents (e.g., number of actors, number of use cases, and number of classes). We employed a stepwise linear regression to build these models. The built models fall in the class of the formal estimation models.

To assess the accuracy of the predictions obtained by applying our models, we have compared their estimations with those obtained considering the models built on software measures (e.g., number of classes, number of files, and number of line of code). The results of such a comparison suggest that requirements measures can effectively employed to estimate software project and product measures of a mobile app. One of the most important practical implications is that we can perform estimation early in the software development process.

Due to the preliminary nature of our study, several possible directions for future work are possible. The most important ones have been discussed in the threats to validity section. As a further future work, we plan to gather data in software projects that simulate in a different way how mobile apps are designed and developed. For example, we are going to ask students to develop mobile apps by applying agile methodologies. This would require the definition of new project measures (e.g., the size of the stories) and the replication of our analyses on the gathered data. Definitively, this future work would improve our awareness on the value of the chosen software project and product measures.

ACKNOWLEDGEMENTS

We thank all the students that took part in the study we have presented in this paper.

REFERENCES

- [1] B. Boehm, C. Abts, and S. Chulani, “Software development cost estimation approaches & a survey,” *Ann. Softw. Eng.*, vol. 10, no. 1–4, pp. 177–205, Jan. 2000.
- [2] L. Briand and I. Wiecek, *Resource Estimation in Software Engineering*. Encyclopedia of Software Engineering, John Wiley & Sons, Inc. All, 2002.
- [3] B. A. Kitchenham, E. Mendes, and G. H. Travassos, “Cross versus within-company cost estimation studies: A systematic review,” *IEEE Trans. on Softw. Eng.*, vol. 33, no. 5, pp. 316–329, 2007.
- [4] M. J. Shepperd and G. F. Kadoda, “Comparing software prediction techniques using simulation,” *IEEE Trans. Software Eng.*, vol. 27, no. 11, pp. 1014–1022, 2001.
- [5] E. Mendes and N. Mosley, “Bayesian network models for web effort prediction: A comparative study,” *IEEE Trans. Softw. Eng.*, vol. 34, no. 6, pp. 723–737, 2008.

- [6] M. E. Joorabchi, A. Mesbah, and P. Kruchten, "Real challenges in mobile app development," in *Proceedings ACM / IEEE International Symposium on Empirical Software Engineering and Measurement*, ACM Press, 2013, pp. 15–24.
- [7] V. Basili, G. Caldiera, and D. H. Rombach, *The Goal Question Metric Paradigm, Encyclopedia of Software Engineering*. John Wiley and Sons, 1994.
- [8] C. Wohlin, P. Runeson, M. Höst, M. Ohlsson, B. Regnell, and A. Wesslén, *Experimentation in Software Engineering*. Springer, 2012.
- [9] GitHub. <https://github.com>.
- [10] B. Bruegge and A. H. Dutoit, *Object-Oriented Software Engineering: Using UML, Patterns and Java, 2nd edition*. Prentice-Hall, 2003.
- [11] I. O. for Standardization, *Information Technology—Software Product Evaluation: Quality Characteristics and Guidelines for their Use, ISO/IEC IS 9126*. Geneva: ISO, 1991.
- [12] V. R. Basili, L. C. Briand, and W. L. Melo, "A validation of object-oriented design metrics as quality indicators," *IEEE Trans. Softw. Eng.*, vol. 22, no. 10, pp. 751–761, 1996.
- [13] T. Gyimóthy, R. Ferenc, and I. Siket, "Empirical validation of object-oriented metrics on open source software for fault prediction," *IEEE Trans. on Softw. Eng.*, vol. 31, no. 10, pp. 897–910, 2005.
- [14] G. Scanniello, C. Gravino, A. Marcus, and T. Menzies, "Class level fault prediction using software clustering," in *Proceedings of International Conference on Automated Software Engineering*. IEEE Computer Society, 2013, pp. 640–645.
- [15] T. J. Hastie and D. Pregibon, *Generalized linear models*, J. M. Chambers and T. J. Hastie, Eds. Wadsworth and Brooks/Cole, 1992.
- [16] S. Geisser, *Predictive Inference: An Introduction*, ser. Chapman and Hall/CRC Monographs on Statistics and Applied Probability Series. Chapman and Hall, 1993.
- [17] L. C. Briand and J. Wüst, "Modeling development effort in object-oriented systems using design properties," *IEEE Trans. on Softw. Eng.*, vol. 27, no. 11, pp. 963–986, 2001.
- [18] E. Kocaguneli, T. Menzies, and J. W. Keung, "Kernel methods for software effort estimation - effects of different kernel functions and bandwidths on estimation accuracy," *Empirical Software Engineering*, vol. 18, no. 1, pp. 1–24, 2013.
- [19] W. J. Conover, *Practical Nonparametric Statistics*, 3rd ed. Wiley, 1998.
- [20] P. Royston, "An extension of Shapiro and Wilk's W test for normality to large samples," *Applied Statistics*, vol. 31, no. 2, pp. 115–124, 1982.
- [21] J. Hannay and M. Jørgensen, "The role of deliberate artificial design elements in software engineering experiments," *IEEE Trans. on Softw. Eng.*, vol. 34, no. 2, pp. 242–259, 2008.
- [22] R. Wieringa and M. Daneva, "Six strategies for generalizing software engineering theories," *Science of computer programming*, vol. 101, pp. 136–152, 2015.
- [23] B. Kitchenham, L. Pickard, and S. L. Pfleeger, "Case studies for method and tool evaluation," *IEEE Software*, pp. 52–62, 1995.
- [24] E. Mendes and B. Kitchenham, "Further Comparison of Cross-company and Within-company Effort Estimation Models for Web Applications," in *Proceedings of International Software Metrics Symposium*. IEEE press, 2004, pp. 348–357.
- [25] L. S. Souza and G. S. Aquino, "Meffortmob: A effort size measurement for mobile application development," *International Journal of Software Engineering & Applications*, vol. 5, no. 4, 2014.
- [26] A. Nitze, A. Schmietendorf, and R. Dumke, "An analogy-based effort estimation approach for mobile application development projects," in *Proceedings of Joint Conference of the International Workshop on Software Measurement and the International Conference on Software Process and Product Measurement*, Oct 2014, pp. 99–103.
- [27] H. van Heeringen and E. Van Gorp, "Measure the functional size of a mobile app: Using the cosmic functional size measurement method," in *Proceedings of Joint Conference of the International Workshop on Software Measurement and the International Conference on Software Process and Product Measurement*, Oct 2014, pp. 11–16.
- [28] A. Abran, J. Desharnais, A. Lesterhuis, B. Londeix, R. Meli, P. Morris, S. Oigny, M. O'Neil, T. Rollo, G. Rule, L. Santillo, C. Symons, and H. Toivonen, "The COSMIC Functional Size Measurement Method – Measurement Manual, version 3.0.1," 2008.
- [29] L. D'Avanzo, F. Ferrucci, C. Gravino, and P. Salza, "Cosmic functional measurement of mobile applications and code size estimation," in *Proceedings of Symposium On Applied Computing*. ACM Press, 2015, pp. 1631–1636.